

5.5 Program Correctness

5.5.1 Introduction

Suppose that we have designed an algorithm to solve a problem and have written a program to implement it. How can we be sure that the program always produces the correct answer? After all the bugs have been removed so that the syntax is correct, we can test the program with sample input. It is not correct if an incorrect result is produced for any sample input. But even if the program gives the correct answer for all sample input, it may not always produce the correct answer (unless all possible input has been tested). We need a proof to show that the program *always* gives the correct output.

Program verification, the proof of correctness of programs, uses the rules of inference and proof techniques described in this chapter, including mathematical induction. Because an incorrect program can lead to disastrous results, a large amount of methodology has been constructed for verifying programs. Efforts have been devoted to automating program verification so that it can be carried out using a computer. However, only limited progress has been made toward this goal. Indeed, some mathematicians and theoretical computer scientists argue that it will never be realistic to mechanize the proof of correctness of complex programs.

Some of the concepts and methods used to prove that programs are correct will be introduced in this section. Many different methods have been devised for proving that programs are correct. We will discuss a widely used method for program verification introduced by Tony Hoare in this section; several other methods are also commonly used. Furthermore, we will not develop a complete methodology for program verification in this book. This section is meant to be a brief introduction to the area of program verification, which ties together the rules of logic, proof techniques, and the concept of an algorithm.

5.5.2 Program Verification

A program is said to be **correct** if it produces the correct output for every possible input. A proof that a program is correct consists of two parts. The first part shows that the correct answer is obtained if the program terminates. This part of the proof establishes the **partial correctness** of the program. The second part of the proof shows that the program always terminates.

To specify what it means for a program to produce the correct output, two propositions are used. The first is the **initial assertion**, which gives the properties that the input values must have. The second is the **final assertion**, which gives the properties that the output of the program should have, if the program did what was intended. The appropriate initial and final assertions must be provided when a program is checked.

Definition 1

A program, or program segment, S is said to be *partially correct with respect to* the initial assertion p and the final assertion q if whenever p is true for the input values of S and S terminates, then q is true for the output values of S . The notation $p\{S\}q$ indicates that the program, or program segment, S is partially correct with respect to the initial assertion p and the final assertion q .

Note: The notation $p\{S\}q$ is known as a *Hoare triple*. Tony Hoare introduced the concept of partial correctness.



Note that the notion of partial correctness has nothing to do with whether a program terminates; it focuses only on whether the program does what it is expected to do if it terminates.

A simple example illustrates the concepts of initial and final assertions.

EXAMPLE 1 Show that the program segment
$$\begin{aligned} y &:= 2 \\ z &:= x + y \end{aligned}$$
Extra Examples is correct with respect to the initial assertion $p: x = 1$ and the final assertion $q: z = 3$.

Solution: Suppose that p is true, so that $x = 1$ as the program begins. Then y is assigned the value 2, and z is assigned the sum of the values of x and y , which is 3. Hence, S is correct with respect to the initial assertion p and the final assertion q . Thus, $p\{S\}q$ is true. 

5.5.3 Rules of Inference

A useful rule of inference proves that a program is correct by splitting the program into a sequence of subprograms and then showing that each subprogram is correct.

Suppose that the program S is split into subprograms S_1 and S_2 . Write $S = S_1; S_2$ to indicate that S is made up of S_1 followed by S_2 . Suppose that the correctness of S_1 with respect to the initial assertion p and final assertion q , and the correctness of S_2 with respect to the initial assertion q and the final assertion r , have been established. It follows that if p is true and S_1 is executed and terminates, then q is true; and if q is true, and S_2 executes and terminates, then r is true. Thus, if p is true and $S = S_1; S_2$ is executed and terminates, then r is true. This rule of inference, called the **composition rule**, can be stated as

$$\frac{\begin{array}{l} p\{S_1\}q \\ q\{S_2\}r \end{array}}{\therefore p\{S_1; S_2\}r}.$$

This rule of inference will be used later in this section.

Next, some rules of inference for program segments involving conditional statements and loops will be given. Because programs can be split into segments for proofs of correctness, this will let us verify many different programs.

5.5.4 Conditional Statements

First, rules of inference for conditional statements will be given. Suppose that a program segment has the form

if *condition* **then**
 S

where S is a block of statements. Then S is executed if *condition* is true, and it is not executed when *condition* is false. To verify that this segment is correct with respect to the initial assertion p and final assertion q , two things must be done. First, it must be shown that when p is true and *condition* is also true, then q is true after S terminates. Second, it must be shown that when p is true and *condition* is false, then q is true (because in this case S does not execute).

This leads to the following rule of inference:

$$\frac{\begin{array}{l} (p \wedge \text{condition})\{S\}q \\ (p \wedge \neg \text{condition}) \rightarrow q \end{array}}{\therefore p\{\text{if } \text{condition} \text{ then } S\}q}.$$

Example 2 illustrates how this rule of inference is used.

EXAMPLE 2 Verify that the program segment

```

if  $x > y$  then
     $y := x$ 

```

is correct with respect to the initial assertion **T** and the final assertion $y \geq x$.

Solution: When the initial assertion is true and $x > y$, the assignment $y := x$ is carried out. Hence, the final assertion, which asserts that $y \geq x$, is true in this case. Moreover, when the initial assertion is true and $x > y$ is false, so that $x \leq y$, the final assertion is again true. Hence, using the rule of inference for program segments of this type, this program is correct with respect to the given initial and final assertions. ▶

Similarly, suppose that a program has a statement of the form

```

if condition then
     $S_1$ 
else
     $S_2$ 

```

If *condition* is true, then S_1 executes; if *condition* is false, then S_2 executes. To verify that this program segment is correct with respect to the initial assertion p and the final assertion q , two things must be done. First, it must be shown that when p is true and *condition* is true, then q is true after S_1 terminates. Second, it must be shown that when p is true and *condition* is false, then q is true after S_2 terminates. This leads to the following rule of inference:

$$\frac{(p \wedge \text{condition})\{S_1\}q \quad (p \wedge \neg \text{condition})\{S_2\}q}{\therefore p\{\text{if condition then } S_1 \text{ else } S_2\}q.}$$

Links ▶



Courtesy of Tony Hoare

C. ANTHONY R. HOARE (BORN 1934) Tony Hoare was born in Colombo, Ceylon (now known as Sri Lanka), where his father was a civil servant of the British Empire and his mother's father owned a plantation. He spent his early childhood in Ceylon, moving to England in 1945. Hoare studied philosophy, together with the classics, at the University of Oxford, where he became interested in computing as a result of his fascination with the power of mathematical logic and the certainty of mathematical truth. He received his bachelors degree from Oxford in 1956.

Hoare learned Russian during his service in the Royal Navy, and later studied the computer translation of natural languages at Moscow State University. He returned to England in 1960, taking a job at a small computer manufacturer, where he wrote a compiler for the programming language Algol. In 1968, he became Professor of Computing Science at the Queen's University, Belfast; in 1977, he moved to the

University of Oxford as Professor of Computing; he is now Professor Emeritus. He is a Fellow of the Royal Society and also holds a position at Microsoft Research in Cambridge.

Hoare has made many contributions to the theory of programming languages and to programming methodology. He was first to define a programming language based on how programs could be proved correct with respect to their specifications. Hoare also invented quick sort, one of the most commonly used sorting algorithms (see the preamble to Exercise 50 in Section 5.4). He received the ACM Turing Award in 1980 and in 2000 he was knighted for services to education and computer science. Hoare is a noted writer in the technical and social aspects of computer science.

Example 3 illustrates how this rule of inference is used.

EXAMPLE 3 Verify that the program segment

```

if  $x < 0$  then
     $abs := -x$ 
else
     $abs := x$ 

```

is correct with respect to the initial assertion **T** and the final assertion $abs = |x|$.

Solution: Two things must be demonstrated. First, it must be shown that if the initial assertion is true and $x < 0$, then $abs = |x|$. This is correct, because when $x < 0$ the assignment statement $abs := -x$ sets $abs = -x$, which is $|x|$ by definition when $x < 0$. Second, it must be shown that if the initial assertion is true and $x < 0$ is false, so that $x \geq 0$, then $abs = |x|$. This is also correct, because in this case the program uses the assignment statement $abs := x$, and x is $|x|$ by definition when $x \geq 0$, so $abs := x$. Hence, using the rule of inference for program segments of this type, this segment is correct with respect to the given initial and final assertions. ◀

5.5.5 Loop Invariants



Next, proofs of correctness of **while** loops will be described. To develop a rule of inference for program segments of the type

```

while condition
    S

```

note that *S* is repeatedly executed until *condition* becomes false. An assertion that remains true each time *S* is executed must be chosen. Such an assertion is called a **loop invariant**. In other words, *p* is a loop invariant if $(p \wedge \text{condition})\{S\}p$ is true.

Suppose that *p* is a loop invariant. It follows that if *p* is true before the program segment is executed, *p* and $\neg \text{condition}$ are true after termination, if it occurs. This rule of inference is

$$\frac{(p \wedge \text{condition})\{S\}p}{\therefore p\{\text{while } \text{condition } S\}(\neg \text{condition} \wedge p)}.$$

The use of a loop invariant is illustrated in Example 4.

EXAMPLE 4 A loop invariant is needed to verify that the program segment



```

i := 1
factorial := 1
while  $i < n$ 
     $i := i + 1$ 
     $\text{factorial} := \text{factorial} \cdot i$ 

```

terminates with $\text{factorial} = n!$ when *n* is a positive integer.

Let p be the assertion “ $factorial = i!$ and $i \leq n$.” We first prove that p is a loop invariant. Suppose that, at the beginning of one execution of the **while** loop, p is true and the condition of the **while** loop holds; in other words, assume that $factorial = i!$ and that $i < n$. The new values i_{new} and $factorial_{\text{new}}$ of i and $factorial$ are $i_{\text{new}} = i + 1$ and $factorial_{\text{new}} = factorial \cdot (i + 1) = (i + 1)! = i_{\text{new}}!$. Because $i < n$, we also have $i_{\text{new}} = i + 1 \leq n$. Thus, p is true at the end of the execution of the loop. This shows that p is a loop invariant.

Now we consider the program segment. Just before entering the loop, $i = 1 \leq n$ and $factorial = 1 = 1! = i!$ both hold, so p is true. Because p is a loop invariant, the rule of inference just introduced implied that if the **while** loop terminates, it terminates with p true and with $i < n$ false. In this case, at the end, $factorial = i!$ and $i \leq n$ are true, but $i < n$ is false; in other words, $i = n$ and $factorial = i! = n!$, as desired.

Finally, we need to check that the **while** loop actually terminates. At the beginning of the program i is assigned the value 1, so after $n - 1$ traversals of the loop, the new value of i will be n , and the loop terminates at that point. 

A final example will be given to show how the various rules of inference can be used to verify the correctness of a longer program.

EXAMPLE 5 We will outline how to verify the correctness of the program S for computing the product of two integers.

```

procedure multiply( $m, n$ : integers)

   $S_1 \left\{ \begin{array}{l} \text{if } n < 0 \text{ then } a := -n \\ \text{else } a := n \end{array} \right.$ 

   $S_2 \left\{ \begin{array}{l} k := 0 \\ x := 0 \end{array} \right.$ 

   $S_3 \left\{ \begin{array}{l} \text{while } k < a \\ \quad x := x + m \\ \quad k := k + 1 \end{array} \right.$ 

   $S_4 \left\{ \begin{array}{l} \text{if } n < 0 \text{ then } product := -x \\ \text{else } product := x \end{array} \right.$ 

  return product
  {product equals  $mn$ }

```

The goal is to prove that after S is executed, $product$ has the value mn . The proof of correctness can be carried out by splitting S into four segments, with $S = S_1; S_2; S_3; S_4$, as shown in the listing of S . The rule of composition can be used to build the correctness proof. Here is how the argument proceeds. The details will be left as an exercise for the reader.

Let p be the initial assertion “ m and n are integers.” Then, it can be shown that $p\{S_1\}q$ is true, when q is the proposition $p \wedge (a = |n|)$. Next, let r be the proposition $q \wedge (k = 0) \wedge (x = 0)$. It is easily verified that $q\{S_2\}r$ is true. It can be shown that “ $x = mk$ and $k \leq a$ ” is an invariant for the loop in S_3 . Furthermore, it is easy to see that the loop terminates after a iterations, with $k = a$, so $x = ma$ at this point. Because r implies that $x = m \cdot 0$ and $0 \leq a$, the loop invariant is true before the loop is entered. Because the loop terminates with $k = a$, it follows that $r\{S_3\}s$ is true, where s is the proposition “ $x = ma$ and $a = |n|$.” Finally, it can be shown that S_4 is correct with respect to the initial assertion s and final assertion t , where t is the proposition “ $product = mn$.”

Putting all this together, because $p\{S_1\}q$, $q\{S_2\}r$, $r\{S_3\}s$, and $s\{S_4\}t$ are all true, it follows from the rule of composition that $p\{S\}t$ is true. Furthermore, because all four segments terminate, S does terminate. This verifies the correctness of the program. ◀

Exercises

1. Prove that the program segment

```
y := 1
z := x + y
```

is correct with respect to the initial assertion $x = 0$ and the final assertion $z = 1$.

2. Verify that the program segment

```
if x < 0 then x := 0
```

is correct with respect to the initial assertion **T** and the final assertion $x \geq 0$.

3. Verify that the program segment

```
x := 2
z := x + y
if y > 0 then
  z := z + 1
else
  z := 0
```

is correct with respect to the initial assertion $y = 3$ and the final assertion $z = 6$.

4. Verify that the program segment

```
if x < y then
  min := x
else
  min := y
```

is correct with respect to the initial assertion **T** and the final assertion $(x \leq y \wedge \text{min} = x) \vee (x > y \wedge \text{min} = y)$.

- *5. Devise a rule of inference for verification of partial correctness of statements of the form

```
if condition 1 then
  S1
else if condition 2 then
  S2
  ⋮
else
  Sn
```

where $S_1, S_2, \dots, S_n$ are blocks.

6. Use the rule of inference developed in Exercise 5 to verify that the program

```
if x < 0 then
  y := -2|x|/x
else if x > 0 then
  y := 2|x|/x
else if x = 0 then
  y := 2
```

is correct with respect to the initial assertion **T** and the final assertion $y = 2$.

7. Use a loop invariant to prove that the following program segment for computing the n th power, where n is a positive integer, of a real number x is correct.

```
power := 1
i := 1
while i ≤ n
  power := power * x
  i := i + 1
```

- *8. Prove that the iterative program for finding f_n given in Section 5.4 is correct.

9. Provide all the details in the proof of correctness given in Example 5.

10. Suppose that both the conditional statement $p_0 \rightarrow p_1$ and the program assertion $p_1\{S\}q$ are true. Show that $p_0\{S\}q$ also must be true.

11. Suppose that both the program assertion $p\{S\}q_0$ and the conditional statement $q_0 \rightarrow q_1$ are true. Show that $p\{S\}q_1$ also must be true.

12. This program computes quotients and remainders.

```
r := a
q := 0
while r ≥ d
  r := r - d
  q := q + 1
```

Verify that it is partially correct with respect to the initial assertion “ a and d are positive integers” and the final assertion “ q and r are integers such that $a = dq + r$ and $0 \leq r < d$.”

13. Use a loop invariant to verify that the Euclidean algorithm (Algorithm 1 in Section 4.3) is partially correct with respect to the initial assertion “ a and b are positive integers” and the final assertion “ $x = \text{gcd}(a, b)$.”

Key Terms and Results

TERMS

sequence: a function with domain that is a subset of the set of integers

geometric progression: a sequence of the form $a, ar, ar^2, \dots$, where a and r are real numbers

arithmetic progression: a sequence of the form $a, a + d, a + 2d, \dots$, where a and d are real numbers

the principle of mathematical induction: The statement $\forall n P(n)$ is true if $P(1)$ is true and $\forall k[P(k) \rightarrow P(k + 1)]$ is true.

basis step: the proof of $P(1)$ in a proof by mathematical induction of $\forall nP(n)$

inductive step: the proof of $P(k) \rightarrow P(k+1)$ for all positive integers k in a proof by mathematical induction of $\forall nP(n)$

strong induction: The statement $\forall nP(n)$ is true if $P(1)$ is true and $\forall k[(P(1) \wedge \cdots \wedge P(k)) \rightarrow P(k+1)]$ is true.

well-ordering property: Every nonempty set of nonnegative integers has a least element.

recursive definition of a function: a definition of a function that specifies an initial set of values and a rule for obtaining values of this function at integers from its values at smaller integers

recursive definition of a set: a definition of a set that specifies an initial set of elements in the set and a rule for obtaining other elements from those in the set

structural induction: a technique for proving results about recursively defined sets

recursive algorithm: an algorithm that proceeds by reducing a problem to the same problem with smaller input

merge sort: a sorting algorithm that sorts a list by splitting it in two, sorting each of the two resulting lists, and merging the results into a sorted list

iteration: a procedure based on the repeated use of operations in a loop

program correctness: verification that a procedure always produces the correct result

loop invariant: a property that remains true during every traversal of a loop

initial assertion: the statement specifying the properties of the input values of a program

final assertion: the statement specifying the properties the output values should have if the program worked correctly

Review Questions

1. a) Can you use the principle of mathematical induction to find a formula for the sum of the first n terms of a sequence?
b) Can you use the principle of mathematical induction to determine whether a given formula for the sum of the first n terms of a sequence is correct?
c) Find a formula for the sum of the first n even positive integers, and prove it using mathematical induction.
2. a) For which positive integers n is $11n + 17 \leq 2^n$?
b) Prove the conjecture you made in part (a) using mathematical induction.
3. a) Which amounts of postage can be formed using only 5-cent and 9-cent stamps?
b) Prove the conjecture you made using mathematical induction.
c) Prove the conjecture you made using strong induction.
d) Find a proof of your conjecture different from the ones you gave in (b) and (c).
4. Give two different examples of proofs that use strong induction.
5. a) State the well-ordering property for the set of positive integers.
b) Use this property to show that every positive integer greater than one can be written as the product of primes.
6. a) Explain why a function f from the set of positive integers to the set of real numbers is well defined if it is defined recursively by specifying $f(1)$ and a rule for finding $f(n)$ from $f(n-1)$.
b) Provide a recursive definition of the function $f(n) = (n+1)!$.
7. a) Give a recursive definition of the Fibonacci numbers.
b) Show that $f_n > \alpha^{n-2}$ whenever $n \geq 3$, where f_n is the n th term of the Fibonacci sequence and $\alpha = (1 + \sqrt{5})/2$.
8. a) Explain why a sequence a_n is well defined if it is defined recursively by specifying a_1 and a_2 and a rule for finding a_n from $a_1, a_2, \dots, a_{n-1}$ for $n = 3, 4, 5, \dots$.
b) Find the value of a_n if $a_1 = 1, a_2 = 2$, and $a_n = a_{n-1} + a_{n-2} + \cdots + a_1$, for $n = 3, 4, 5, \dots$.
9. Give two examples of how well-formed formulae are defined recursively for different sets of elements and operators.
10. a) Give a recursive definition of the length of a string.
b) Use the recursive definition from part (a) and structural induction to prove that $l(xy) = l(x) + l(y)$.
11. a) What is a recursive algorithm?
b) Describe a recursive algorithm for computing the sum of n numbers in a sequence.
12. Describe a recursive algorithm for computing the greatest common divisor of two positive integers.
13. a) Describe the merge sort algorithm.
b) Use the merge sort algorithm to put the list 4, 10, 1, 5, 3, 8, 7, 2, 6, 9 in increasing order.
c) Give a big- O estimate for the number of comparisons used by the merge sort.
14. a) Does testing a computer program to see whether it produces the correct output for certain input values verify that the program always produces the correct output?
b) Does showing that a computer program is partially correct with respect to an initial assertion and a final assertion verify that the program always produces the correct output? If not, what else is needed?
15. What techniques can you use to show that a long computer program is partially correct with respect to an initial assertion and a final assertion?
16. What is a loop invariant? How is a loop invariant used?

Supplementary Exercises

- Use mathematical induction to show that $\frac{2}{3} + \frac{2}{9} + \frac{2}{27} + \dots + \frac{2}{3^n} = 1 - \frac{1}{3^n}$ whenever n is a positive integer.
- Use mathematical induction to show that $1^3 + 3^3 + 5^3 + \dots + (2n+1)^3 = (n+1)^2(2n^2 + 4n + 1)$ whenever n is a positive integer.
- Use mathematical induction to show that $1 \cdot 2^0 + 2 \cdot 2^1 + 3 \cdot 2^2 + \dots + n \cdot 2^{n-1} = (n-1) \cdot 2^n + 1$ whenever n is a positive integer.
- Use mathematical induction to show that
$$\frac{1}{1 \cdot 3} + \frac{1}{3 \cdot 5} + \dots + \frac{1}{(2n-1)(2n+1)} = \frac{n}{2n+1}$$
 whenever n is a positive integer.
- Show that
$$\frac{1}{1 \cdot 4} + \frac{1}{4 \cdot 7} + \dots + \frac{1}{(3n-2)(3n+1)} = \frac{n}{3n+1}$$
 whenever n is a positive integer.
- Use mathematical induction to show that $2^n > n^2 + n$ whenever n is an integer greater than 4.
- Use mathematical induction to show that $2^n > n^3$ whenever n is an integer greater than 9.
- Find an integer N such that $2^n > n^4$ whenever n is an integer greater than N . Prove that your result is correct using mathematical induction.
- Use mathematical induction to prove that $a - b$ is a factor of $a^n - b^n$ whenever n is a positive integer.
- Use mathematical induction to prove that 9 divides $n^3 + (n+1)^3 + (n+2)^3$ whenever n is a nonnegative integer.
- Use mathematical induction to prove that 43 divides $6^{n+1} + 7^{2n-1}$ for every positive integer n .
- Use mathematical induction to prove that 64 divides $3^{2n+2} + 56n + 55$ for every positive integer n .
- Use mathematical induction to prove this formula for the sum of the terms of an arithmetic progression.
$$a + (a+d) + \dots + (a+nd) = (n+1)(2a+nd)/2$$
- Suppose that $a_j \equiv b_j \pmod{m}$ for $j = 1, 2, \dots, n$. Use mathematical induction to prove that
 - $\sum_{j=1}^n a_j \equiv \sum_{j=1}^n b_j \pmod{m}$.
 - $\prod_{j=1}^n a_j \equiv \prod_{j=1}^n b_j \pmod{m}$.
- Show that if n is a positive integer, then
$$\sum_{k=1}^n \frac{k+4}{k(k+1)(k+2)} = \frac{n(3n+7)}{2(n+1)(n+2)}.$$
- For which positive integers n is $n+6 < (n^2-8n)/16$? Prove your answer using mathematical induction.
- (Requires calculus) Suppose that $f(x) = e^x$ and $g(x) = xe^x$. Use mathematical induction together with the product rule and the fact that $f'(x) = e^x$ to prove that $g^{(n)}(x) = (x+n)e^x$ whenever n is a positive integer.
- (Requires calculus) Suppose that $f(x) = e^x$ and $g(x) = e^{cx}$, where c is a constant. Use mathematical induction together with the chain rule and the fact that $f'(x) = e^x$ to prove that $g^{(n)} = c^n e^{cx}$ whenever n is a positive integer.
- * Formulate a conjecture about which Fibonacci numbers are even, and use a form of mathematical induction to prove your conjecture.
- * Determine which Fibonacci numbers are divisible by 3. Use a form of mathematical induction to prove your conjecture.
- * Prove that $f_k f_n + f_{k+1} f_{n+1} = f_{n+k+1}$ for all nonnegative integers n and k , where f_i denotes the i th Fibonacci number. Recall from Example 15 of Section 2.4 that the sequence of **Lucas numbers** is defined by $l_0 = 2$, $l_1 = 1$, and $l_n = l_{n-1} + l_{n-2}$ for $n = 2, 3, 4, \dots$.
- Show that $f_n + f_{n+2} = l_{n+1}$ whenever n is a positive integer, where f_i and l_i are the i th Fibonacci number and i th Lucas number, respectively.
- Show that $l_0^2 + l_1^2 + \dots + l_n^2 = l_n l_{n+1} + 2$ whenever n is a nonnegative integer and l_i is the i th Lucas number.
- * Use mathematical induction to show that the product of any n consecutive positive integers is divisible by $n!$. [Hint: Use the identity $m(m+1) \dots (m+n-1)/n! = (m-1)m(m+1) \dots (m+n-2)/n! + m(m+1) \dots (m+n-2)/(n-1)!]$
- Use mathematical induction to show that $(\cos x + i \sin x)^n = \cos nx + i \sin nx$ whenever n is a positive integer. (Here i is the square root of -1 .) [Hint: Use the identities $\cos(a+b) = \cos a \cos b - \sin a \sin b$ and $\sin(a+b) = \sin a \cos b + \cos a \sin b$.]
- * Use mathematical induction to show that $\sum_{j=1}^n \cos jx = \cos[(n+1)x/2] \sin(nx/2) / \sin(x/2)$ whenever n is a positive integer and $\sin(x/2) \neq 0$.
- Use mathematical induction to prove that $\sum_{j=1}^n j^2 2^j = n^2 2^{n+1} - n 2^{n+2} + 3 \cdot 2^{n+1} - 6$ for every positive integer n .
- (Requires calculus) Suppose that the sequence $x_1, x_2, \dots, x_n, \dots$ is recursively defined by $x_1 = 0$ and $x_{n+1} = \sqrt{x_n + 6}$.
 - Use mathematical induction to show that $x_1 < x_2 < \dots < x_n < \dots$, that is, the sequence $\{x_n\}$ is monotonically increasing.
 - Use mathematical induction to prove that $x_n < 3$ for $n = 1, 2, \dots$.
 - Show that $\lim_{n \rightarrow \infty} x_n = 3$.
- Show if n is a positive integer with $n \geq 2$, then
$$\sum_{j=2}^n \frac{1}{j^2 - 1} = \frac{(n-1)(3n+2)}{4n(n+1)}.$$

30. Use mathematical induction to prove Theorem 1 in Section 4.2, that is, show if b is an integer, where $b > 1$, and n is a positive integer, then n can be expressed uniquely in the form $n = a_k b^k + a_{k-1} b^{k-1} + \cdots + a_1 b + a_0$.
- *31. A **lattice point** in the plane is a point (x, y) where both x and y are integers. Use mathematical induction to show that at least $n + 1$ straight lines are needed to ensure that every lattice point (x, y) with $x \geq 0$, $y \geq 0$, and $x + y \leq n$ lies on one of these lines.
32. (Requires calculus) Use mathematical induction and the product rule to show that if n is a positive integer and $f_1(x), f_2(x), \dots, f_n(x)$, are all differentiable functions, then
- $$\frac{(f_1(x)f_2(x) \cdots f_n(x))'}{f_1(x)f_2(x) \cdots f_n(x)} = \frac{f_1'(x)}{f_1(x)} + \frac{f_2'(x)}{f_2(x)} + \cdots + \frac{f_n'(x)}{f_n(x)}.$$
33. (Requires material in Section 2.6) Suppose that $\mathbf{B} = \mathbf{M}\mathbf{A}\mathbf{M}^{-1}$, where $\mathbf{A}$ and $\mathbf{B}$ are $n \times n$ matrices and $\mathbf{M}$ is invertible. Show that $\mathbf{B}^k = \mathbf{M}\mathbf{A}^k\mathbf{M}^{-1}$ for all positive integers k . (Consult both the text of Section 2.6 and the preamble to Exercise 18 of Section 2.6.)
34. Use mathematical induction to show that if you draw lines in the plane you only need two colors to color the regions formed so that no two regions that have an edge in common have a common color.
35. Show that $n!$ can be represented as the sum of n of its distinct positive divisors whenever $n \geq 3$. [Hint: Use inductive loading. First try to prove this result using mathematical induction. By examining where your proof fails, find a stronger statement that you can easily prove using mathematical induction.]
- *36. Use mathematical induction to prove that if $x_1, x_2, \dots, x_n$ are positive real numbers with $n \geq 2$, then
- $$\left(x_1 + \frac{1}{x_1}\right)\left(x_2 + \frac{1}{x_2}\right) \cdots \left(x_n + \frac{1}{x_n}\right) \geq \left(x_1 + \frac{1}{x_2}\right)\left(x_2 + \frac{1}{x_3}\right) \cdots \left(x_{n-1} + \frac{1}{x_n}\right)\left(x_n + \frac{1}{x_1}\right)$$
37. Use mathematical induction to prove that if n people stand in a line, where n is a positive integer, and if the first person in the line is a woman and the last person in line is a man, then somewhere in the line there is a woman directly in front of a man.
- *38. Suppose that for every pair of cities in a country there is a direct one-way road connecting them in one direction or the other. Use mathematical induction to show that there is a city that can be reached from every other city either directly or via exactly one other city.
39. Use mathematical induction to show that when n circles divide the plane into regions, these regions can be colored with two different colors such that no regions with a common boundary are colored the same.
- *40. Suppose that among a group of cars on a circular track there is enough fuel for one car to complete a lap. Use mathematical induction to show that there is a car in the

group that can complete a lap by obtaining gas from other cars as it travels around the track.

41. Show that if n is a positive integer, then

$$\sum_{j=1}^n (2j-1) \left(\sum_{k=j}^n 1/k \right) = n(n+1)/2.$$

42. Use mathematical induction to show that if a , b , and c are the lengths of the sides of a right triangle, where c is the length of the hypotenuse, then $a^n + b^n < c^n$ for all integers n with $n \geq 3$.
- *43. Use mathematical induction to show that if n is a positive integer, the sequence $2 \bmod n, 2^2 \bmod n, 2^{2^2} \bmod n, \dots$ is eventually constant (that is, all terms after a finite number of terms are all the same).
44. A **unit** or **Egyptian fraction** is a fraction of the form $1/n$, where n is a positive integer. In this exercise, we will use strong induction to show that a greedy algorithm can be used to express every rational number p/q with $0 < p/q < 1$ as the sum of distinct unit fractions. At each step of the algorithm, we find the smallest positive integer n such that $1/n$ can be added to the sum without exceeding p/q . For example, to express $5/7$ we first start the sum with $1/2$. Because $5/7 - 1/2 = 3/14$ we add $1/5$ to the sum because 5 is the smallest positive integer k such that $1/k < 3/14$. Because $3/14 - 1/5 = 1/70$, the algorithm terminates, showing that $5/7 = 1/2 + 1/5 + 1/70$. Let $T(p)$ be the statement that this algorithm terminates for all rational numbers p/q with $0 < p/q < 1$. We will prove that the algorithm always terminates by showing that $T(p)$ holds for all positive integers p .
- a) Show that the basis step $T(1)$ holds.
- b) Suppose that $T(k)$ holds for positive integers k with $k < p$. That is, assume that the algorithm terminates for all rational numbers k/r , where $1 \leq k < p$. Show that if we start with p/q and the fraction $1/n$ is selected in the first step of the algorithm, then $p/q = p'/q' + 1/n$, where $p' = np - q$ and $q' = nq$. After considering the case where $p/q = 1/n$, use the inductive hypothesis to show that the greedy algorithm terminates when it begins with p'/q' and complete the inductive step.

The **McCarthy 91 function** (defined by John McCarthy, one of the founders of artificial intelligence) is defined using the rule

$$M(n) = \begin{cases} n - 10 & \text{if } n > 100 \\ M(M(n + 11)) & \text{if } n \leq 100 \end{cases}$$

for all positive integers n .

45. By successively using the defining rule for $M(n)$, find

$$\begin{array}{lll} \text{a) } M(102). & \text{b) } M(101). & \text{c) } M(99). \\ \text{d) } M(97). & \text{e) } M(87). & \text{f) } M(76). \end{array}$$

- **46. Show that the function $M(n)$ is a well-defined function from the set of positive integers to the set of positive integers. [Hint: Prove that $M(n) = 91$ for all positive integers n with $n \leq 101$.]

47. Is this proof that

$$\frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \cdots + \frac{1}{(n-1)n} = \frac{3}{2} - \frac{1}{n},$$

whenever n is a positive integer, correct? Justify your answer.

Basis step: The result is true when $n = 1$ because

$$\frac{1}{1 \cdot 2} = \frac{3}{2} - \frac{1}{1}.$$

Inductive step: Assume that the result is true for n . Then

$$\begin{aligned} \frac{1}{1 \cdot 2} + \frac{1}{2 \cdot 3} + \cdots + \frac{1}{(n-1)n} + \frac{1}{n(n+1)} \\ = \frac{3}{2} - \frac{1}{n} + \left(\frac{1}{n} - \frac{1}{n+1} \right) \\ = \frac{3}{2} - \frac{1}{n+1}. \end{aligned}$$

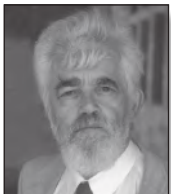
Hence, the result is true for $n + 1$ if it is true for n . This completes the proof.

48. Suppose that $A_1, A_2, \dots, A_n$ are a collection of sets. Suppose that $R_2 = A_1 \oplus A_2$ and $R_k = R_{k-1} \oplus A_k$ for $k = 3, 4, \dots, n$. Use mathematical induction to prove that $x \in R_n$ if and only if x belongs to an odd number of the sets $A_1, A_2, \dots, A_n$. (Recall that $S \oplus T$ is the symmetric difference of the sets S and T defined in the preamble to Exercise 38 of Section 2.2.)
- *49. Show that n circles divide the plane into $n^2 - n + 2$ regions if every two circles intersect in exactly two points and no three circles contain a common point.
- *50. Show that n planes divide three-dimensional space into $(n^3 + 5n + 6)/6$ regions if any three of these planes have exactly one point in common and no four contain a common point.
- *51. Use the well-ordering property to show that $\sqrt{2}$ is irrational. [Hint: Assume that $\sqrt{2}$ is rational. Show that the set of positive integers of the form $b\sqrt{2}$ has a least

element a . Then show that $a\sqrt{2} - a$ is a smaller positive integer of this form.]

52. A set is **well ordered** if every nonempty subset of this set has a least element. Determine whether each of the following sets is well ordered.
- the set of integers
 - the set of integers greater than -100
 - the set of positive rationals
 - the set of positive rationals with denominator less than 100
53. a) Show that if $a_1, a_2, \dots, a_n$ are positive integers, then $\gcd(a_1, a_2, \dots, a_{n-1}, a_n) = \gcd(a_1, a_2, \dots, a_{n-2}, \gcd(a_{n-1}, a_n))$.
b) Use part (a), together with the Euclidean algorithm, to develop a recursive algorithm for computing the greatest common divisor of a set of n positive integers.
- *54. Describe a recursive algorithm for writing the greatest common divisor of n positive integers as a linear combination of these integers.
55. Find an explicit formula for $f(n)$ if $f(1) = 1$ and $f(n) = f(n-1) + 2n - 1$ for $n \geq 2$. Prove your result using mathematical induction.
- **56. Give a recursive definition of the set of bit strings that contain twice as many 0s as 1s.
57. Let S be the set of bit strings defined recursively by $\lambda \in S$ and $0x \in S, x1 \in S$ if $x \in S$, where λ is the empty string.
- Find all strings in S of length not exceeding five.
 - Give an explicit description of the elements of S .
58. Let S be the set of strings defined recursively by $abc \in S$, $bac \in S$, and $acb \in S$, where a, b , and c are fixed letters; and for all $x \in S$, $abxc \in S$, $axbc \in S$, and $xabc \in S$, where x is a variable representing a string of letters.
- Find all elements of S of length eight or less.
 - Show that every element of S has a length divisible by three.

Links



©Matthew Naythons/The LIFE Images Collection/Getty Images

JOHN MCCARTHY (1927–2011) John McCarthy was born in Boston. He grew up in Boston and in Los Angeles. He studied mathematics as both an undergraduate and a graduate student, receiving his B.S. in 1948 from the California Institute of Technology and his Ph.D. in 1951 from Princeton. After graduating from Princeton, McCarthy held positions at Princeton, Stanford, Dartmouth, and M.I.T. He held a position at Stanford from 1962 until 1994, and was an emeritus professor there. At Stanford, he was the director of the Artificial Intelligence Laboratory, held a named chair in the School of Engineering, and was a senior fellow at the Hoover Institution.

McCarthy was a pioneer in the study of artificial intelligence, a term he coined in 1955. He worked on problems related to the reasoning and information needs required for intelligent computer behavior. McCarthy was among the first computer scientists to design time-sharing computer systems. He developed LISP, a programming language for computing using symbolic expressions. He played an important role in using logic to verify the correctness of computer programs. McCarthy has also worked on the social implications of computer technology. In his later years he worked on the problem of how people and computers make conjectures through assumptions that complications are absent from situations. McCarthy was an advocate of the sustainability of human progress and was optimistic about the future of humanity. In his later years he also wrote science fiction stories. Some of his last work explored the possibility that the world is a computer program written by some higher force.

Among the awards McCarthy won are the Turing Award from the Association for Computing Machinery, the Research Excellence Award of the International Conference on Artificial Intelligence, the Kyoto Prize, and the National Medal of Science.

The set B of all **balanced strings of parentheses** is defined recursively by $\lambda \in B$, where λ is the empty string; $(x) \in B$, $xy \in B$ if $x, y \in B$.

59. Show that $((()))$ is a balanced string of parentheses and $(()))$ is not a balanced string of parentheses.
60. Find all balanced strings of parentheses with exactly six symbols.
61. Find all balanced strings of parentheses with four or fewer symbols.
62. Use induction to show that if x is a balanced string of parentheses, then the number of left parentheses equals the number of right parentheses in x .

Define the function N on the set of strings of parentheses by

$$N(\lambda) = 0, N(()) = 1, N(()) = -1, \\ N(uv) = N(u) + N(v),$$

where λ is the empty string, and u and v are strings. It can be shown that N is well defined.

63. Find
 - a) $N(())$.
 - b) $N(())(())$.
 - c) $N((()))$.
 - d) $N(((()))$.
- **64. Show that a string w of parentheses is balanced if and only if $N(w) = 0$ and $N(u) \geq 0$ whenever u is a prefix of w , that is, $w = uv$.
- *65. Give a recursive algorithm for finding all balanced strings of parentheses containing n or fewer symbols.
66. Give a recursive algorithm for finding $\gcd(a, b)$, where a and b are nonnegative integers not both zero, based on these facts: $\gcd(a, b) = \gcd(b, a)$ if $a > b$, $\gcd(0, b) = b$, $\gcd(a, b) = 2 \gcd(a/2, b/2)$ if a and b are even, $\gcd(a, b) = \gcd(a/2, b)$ if a is even and b is odd, and $\gcd(a, b) = \gcd(a, b - a)$.
67. Verify the program segment


```
if  $x > y$  then
   $x := y$ 
```

 with respect to the initial assertion **T** and the final assertion $x \leq y$.
- *68. Develop a rule of inference for verifying recursive programs and use it to verify the recursive algorithm for computing factorials given as Algorithm 1 in Section 5.4.

69. Devise a recursive algorithm that counts the number of times the integer 0 occurs in a list of integers.

Exercises 70–77 deal with some unusual sequences, informally called **self-generating sequences**, produced by simple recurrence relations or rules. In particular, Exercises 70–75 deal with the sequence $\{a(n)\}$ defined by $a(n) = n - a(a(n-1))$ for $n \geq 1$ and $a(0) = 0$. (This sequence, as well as those in Exercises 74 and 75, are defined in Douglas Hofstadter's fascinating book *Gödel, Escher, Bach* ([Ho99]).

70. Find the first 10 terms of the sequence $\{a(n)\}$ defined in the preamble to this exercise.
 - *71. Prove that this sequence is well defined. That is, show that $a(n)$ is uniquely defined for all nonnegative integers n .
 - **72. Prove that $a(n) = \lfloor (n+1)\mu \rfloor$ where $\mu = (-1 + \sqrt{5})/2$. [Hint: First show for all $n > 0$ that $(\mu n - \lfloor \mu n \rfloor) + (\mu^2 n - \lfloor \mu^2 n \rfloor) = 1$. Then show for all real numbers α with $0 \leq \alpha < 1$ and $\alpha \neq 1 - \mu$ that $\lfloor (1 + \mu)(1 - \alpha) \rfloor + \lfloor \alpha + \mu \rfloor = 1$, considering the cases $0 \leq \alpha < 1 - \mu$ and $1 - \mu < \alpha < 1$ separately.]
 - *73. Use the formula from Exercise 72 to show that $a(n) = a(n-1)$ if $\mu n - \lfloor \mu n \rfloor < 1 - \mu$ and $a(n) = a(n-1) + 1$ otherwise.
 74. Find the first 10 terms of each of the following self-generating sequences:
 - a) $a(n) = n - a(a(n-1))$ for $n \geq 1$, $a(0) = 0$
 - b) $a(n) = n - a(a(a(n-1)))$ for $n \geq 1$, $a(0) = 0$
 - c) $a(n) = a(n - a(n-1)) + a(n - a(n-2))$ for $n \geq 3$, $a(1) = 1$ and $a(2) = 1$
 75. Find the first 10 terms of both the sequences $m(n)$ and $f(n)$ defined by the following pair of interwoven recurrence relations: $m(n) = n - f(m(n-1))$, $f(n) = n - m(f(n-1))$ for $n \geq 1$, $f(0) = 1$ and $m(0) = 0$.
- Golomb's self-generating sequence** is the unique nondecreasing sequence of positive integers $a_1, a_2, a_3, \dots$ that has the property that it contains exactly a_k occurrences of k for each positive integer k .
76. Find the first 20 terms of Golomb's self-generating sequence.
 - *77. Show that if $f(n)$ is the largest integer m such that $a_m = n$, where a_m is the m th term of Golomb's self-generating sequence, then $f(n) = \sum_{k=1}^n a_k$ and $f(f(n)) = \sum_{k=1}^n ka_k$.

Computer Projects

Write programs with these input and output.

- **1. Given a $2^n \times 2^n$ checkerboard with one square missing, construct a tiling of this checkerboard using right triominoes.
- **2. Generate all well-formed formulae for expressions involving the variables x, y , and z and the operators $\{+, *, /, -\}$ with n or fewer symbols.
- **3. Generate all well-formed formulae for propositions with n or fewer symbols where each symbol is **T**, **F**, one of

the propositional variables p and q , or an operator from $\{\neg, \vee, \wedge, \rightarrow, \leftrightarrow\}$.

4. Given a string, find its reversal.
5. Given a real number a and a nonnegative integer n , find a^n using recursion.
6. Given a real number a and a nonnegative integer n , find a^{2^n} using recursion.

- *7. Given a real number a and a nonnegative integer n , find a^n using the binary expansion of n and a recursive algorithm for computing a^{2^k} .
- 8. Given two integers not both zero, find their greatest common divisor using recursion.
- 9. Given a list of integers and an element x , locate x in this list using a recursive implementation of a linear search.
- 10. Given a list of integers and an element x , locate x in this list using a recursive implementation of a binary search.
- 11. Given a nonnegative integer n , find the n th Fibonacci number using iteration.
- 12. Given a nonnegative integer n , find the n th Fibonacci number using recursion.
- 13. Given a positive integer, find the number of partitions of this integer. (See Exercise 49 of Section 5.3.)
- 14. Given positive integers m and n , find $A(m, n)$, the value of Ackermann's function at the pair (m, n) . (See the preamble to Exercise 50 of Section 5.3.)
- 15. Given a list of integers, sort these integers using the merge sort.

Computations and Explorations

Use a computational program or programs you have written to do these exercises.

- 1. What are the largest values of n for which $n!$ has fewer than 100 decimal digits and fewer than 1000 decimal digits?
- 2. Determine which Fibonacci numbers are divisible by 5, which are divisible by 7, and which are divisible by 11. Prove that your conjectures are correct.
- 3. Construct tilings using right triominoes of various 16×16 , 32×32 , and 64×64 checkerboards with one square missing.
- 4. Explore which $m \times n$ checkerboards can be completely covered by right triominoes. Can you make a conjecture that answers this question?
- **5. Implement an algorithm for determining whether a point is in the interior or exterior of a simple polygon.
- **6. Implement an algorithm for triangulating a simple polygon.
- 7. Which values of Ackermann's function are small enough that you are able to compute them?
- 8. Compare either the number of operations or the time needed to compute Fibonacci numbers recursively versus that needed to compute them iteratively.

Writing Projects

Respond to these with essays using outside sources.

- 1. Describe the origins of mathematical induction. Who were the first people to use it and to which problems did they apply it?
- 2. Explain how to prove the Jordan curve theorem for simple polygons and describe an algorithm for determining whether a point is in the interior or exterior of a simple polygon.
- 3. Describe how the triangulation of simple polygons is used in some key algorithms in computational geometry.
- 4. Describe a variety of different applications of the Fibonacci numbers to the biological and the physical sciences.
- 5. Discuss the uses of Ackermann's function both in the theory of recursive definitions and in the analysis of the complexity of algorithms for set unions.
- 6. Give the recursive definition of Knuth's up-arrow notation and illustrate its use with a variety of examples, including how it can be used to express values of the Ackermann function (defined in the preamble of Exercise 50 in Section 5.3).
- 7. Discuss some of the various methodologies used to establish the correctness of programs and compare them to Hoare's methods described in Section 5.5.
- 8. Explain how the ideas and concepts of program correctness can be extended to prove that operating systems are secure.